

Pipeline de Dados com IoT e Docker

Relatório técnico · Parte Teórica

DISCIPLINA

Disruptive Architectures: IoT, Big Data e IA

ALUNO

Kaique Ferreira Melo

REPOSITÓRIO

github.com/KaiqueDev1/iot-faculdade-setembro

DATA

Setembro de 2026

Sumário

1. Contextualização do projeto
 2. Arquitetura da solução
 3. Passos realizados
 4. Views SQL
 5. Visualizações no Streamlit
 6. Insights obtidos
 7. Uso prático em ambiente real
 8. Conclusão
- Referências

1. Contextualização do projeto

Dispositivos de Internet das Coisas (IoT), como sensores de temperatura, geram leituras contínuas e em grande volume. Sozinhos, esses dados brutos pouco ajudam: ficam espalhados em arquivos, com datas em formato de texto, registros duplicados e nenhuma forma simples de consulta. Para que virem informação útil (alertas, tendências, decisões de climatização), é preciso um **pipeline de dados** que colete, trate, armazene e apresente essas leituras.

Problema: transformar cerca de 97 mil leituras de temperatura de sensores IoT, disponíveis em um CSV público, em uma base consultável e em um painel visual que permita entender o comportamento térmico do ambiente monitorado.

Proposta: construir um pipeline completo e reproduzível com **Python** (tratamento e carga), **PostgreSQL em Docker** (armazenamento), **views SQL** (camada de análise) e **Streamlit + Plotly** (dashboard interativo). Todo o código está versionado em um repositório público no GitHub.

Base de dados utilizada

Dataset *Temperature Readings: IoT Devices* (Kaggle). As leituras vão de 28/07/2018 a 08/12/2018 e vêm de dois sensores de uma mesma sala (*Room Admin*): um interno (**In**) e um externo (**Out**).

Coluna no CSV	Coluna no banco	Descrição
id	reading_id	Identificador único da leitura
room_id/id	room_id	Sala onde o sensor está instalado
noted_date	noted_date	Data/hora da leitura (texto dd-mm-aaaa hh:mm convertido para TIMESTAMP)
temp	temperature	Temperatura em °C
out/in	location	In = sensor interno · Out = sensor externo

2. Arquitetura da solução

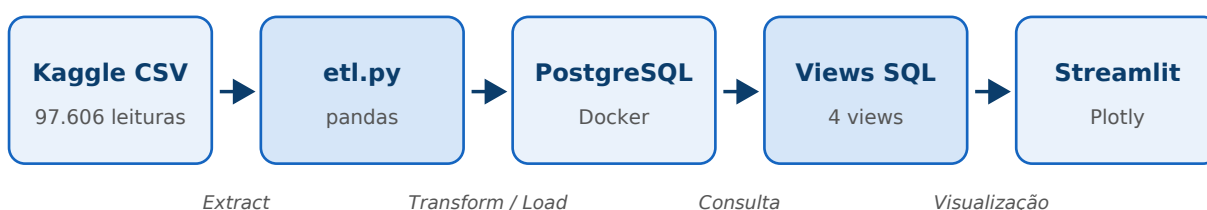


Figura 1 – Fluxo do pipeline: do CSV do Kaggle até o dashboard.

Tecnologia	Função no projeto
Python 3 + pandas	Leitura do CSV, limpeza e transformação dos dados
SQLAlchemy + psycopg2	Conexão do Python com o PostgreSQL
Docker / Docker Compose	Execução do PostgreSQL em contêiner isolado e reproduzível
PostgreSQL 16	Armazenamento da tabela e das views de análise
Streamlit + Plotly	Dashboard web com gráficos interativos
python-dotenv	Credenciais do banco lidas do arquivo .env (fora do código)
Git + GitHub	Versionamento e publicação do projeto

Organização do repositório

```
iot-pipeline/  
├── data/temperature_readings.csv    # dataset do Kaggle  
├── docs/screenshots/               # prints do dashboard  
├── docs/relatorio/                 # este relatório (PDF)  
├── sql/schema.sql                  # criação da tabela  
├── sql/view.sql                    # views de análise  
├── src/config.py                   # conexão com o banco (.env)  
├── src/etl.py                      # extração, tratamento e carga  
├── src/create_view.py              # cria as views no PostgreSQL  
├── src/app.py                      # dashboard Streamlit  
├── docker-compose.yml              # contêiner PostgreSQL  
├── requirements.txt / .env.example / .gitignore  
└── README.md
```

3. Passos realizados

3.1 Configuração do ambiente

Foram instalados o Python, o Docker Desktop e o Git. O Git foi configurado com nome e e-mail do autor, e o projeto foi iniciado como repositório local (`git init`) ligado a um repositório remoto público no GitHub.

3.2 Instalação das dependências

As bibliotecas estão listadas no `requirements.txt` e são instaladas em um ambiente virtual:

```
python -m venv venv
venv\Scripts\activate
# pandas, psycopg2-binary, sqlalchemy, streamlit, plotly, python-dotenv
pip install -r requirements.txt
```

3.3 Criação do contêiner Docker

O PostgreSQL roda em um contêiner definido no `docker-compose.yml`, com volume persistente (os dados sobrevivem à recriação do contêiner) e *healthcheck*. As credenciais vêm do arquivo `.env`, que não é enviado ao GitHub.

```
services:
  postgres:
    image: postgres:16
    container_name: postgres-iot
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-postgres}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-admin}
      POSTGRES_DB: ${POSTGRES_DB:-iot_db}
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
```

```
docker compose up -d      # sobe o banco
docker ps                 # confere o contêiner postgres-iot
```

3.4 Processamento e inserção no banco (ETL)

O script `src/etl.py` segue as três etapas clássicas de um ETL:

- **Extract:** leitura do CSV com pandas (97.606 linhas).
- **Transform:** renomeação das colunas para nomes padronizados, conversão de `noted_date` de texto (dd-mm-aaaa hh:mm) para data/hora real e remoção de registros nulos ou com id duplicado (1 duplicado encontrado).
- **Load:** execução do `sql/schema.sql`, que recria a tabela com tipos corretos, chave primária e índices, seguida da inserção em lotes de 5.000 linhas e de uma contagem de conferência.

```
df["noted_date"] = pd.to_datetime(df["noted_date"], format="%d-%m-%Y %H:%M")
df = df.dropna().drop_duplicates(subset="reading_id")
run_sql_file(engine, "schema.sql")
df.to_sql("temperature_readings", engine, if_exists="append",
         index=False, chunksize=5000, method="multi")
```

```
> python src/etl.py
[extract] 97606 linhas lidas de temperature_readings.csv
[transform] 1 linha(s) removida(s) (nulos/duplicados)
[load] 97605 linhas gravadas no PostgreSQL
```

3.5 Criação das views SQL

As views ficam em um único arquivo, `sql/view.sql`, executado por `src/create_view.py`. Assim, o SQL versionado no GitHub é exatamente o que roda no banco. O script também confere quantas linhas cada view retorna:

```
> python src/create_view.py
vw_resumo_dispositivo: 2 linha(s)
vw_anomalias: 2400 linha(s)
vw_tendencia: 86 linha(s)
vw_leituras_por_hora: 48 linha(s)
```

3.6 Construção do dashboard

O `src/app.py` consulta as views com pandas e gera os gráficos com Plotly Express. A conexão é criada uma vez por sessão (`st.cache_resource`) e os dados ficam em cache por 5 minutos (`st.cache_data`). O painel mostra indicadores gerais e quatro gráficos interativos (zoom, filtro por legenda e detalhes ao passar o mouse).

```
python -m streamlit run src/app.py    # abre em http://localhost:8501
```

3.7 Versionamento com Git

```
git init
git add .
git commit -m "feat: commit inicial do projeto IoT"
git branch -M main
git remote add origin https://github.com/KaiqueDev1/iot-faculdade-setembro.git
git push -u origin main
git rm --cached .env          # remove credenciais do versionamento
git pull / git push           # sincronização a cada etapa
```

4. Views SQL

Foram criadas três views principais, exigidas no projeto, e uma quarta view complementar. Cada uma responde a uma pergunta de negócio.

View 1 - vw_resumo_dispositivo

Pergunta: como cada sensor se comporta de forma geral? **Propósito:** resumir, por dispositivo (In/Out), a quantidade de leituras e as temperaturas média, mínima e máxima. É a visão consolidada que permite comparar o ambiente interno com o externo.

```
CREATE OR REPLACE VIEW vw_resumo_dispositivo AS
SELECT room_id, location AS device_id,
       COUNT(*) AS total_leituras,
       ROUND(AVG(temperature), 2) AS temp_media,
       MIN(temperature) AS temp_min, MAX(temperature) AS temp_max
FROM temperature_readings
GROUP BY room_id, location;
```

Dispositivo	Leituras	Média (°C)	Mín (°C)	Máx (°C)
In (interno)	20.345	30,45	21	41
Out (externo)	77.260	36,27	24	51

View 2 - vw_anomalias

Pergunta: quais leituras fogem do normal? **Propósito:** identificar leituras que se afastam mais de **2 desvios-padrão** da média do **próprio ambiente** (z-score). Esse critério é melhor que um limite fixo: 40 °C é comum no sensor externo, mas é um pico no interno. A view calcula a média e o desvio de cada ambiente numa CTE e devolve cada leitura anômala com seu z-score. Resultado: **2.400 leituras** (2,5% do total), 967 internas e 1.433 externas.

```
CREATE OR REPLACE VIEW vw_anomalias AS
WITH estatisticas AS (
  SELECT location, AVG(temperature) AS media, STDDEV(temperature) AS desvio
  FROM temperature_readings GROUP BY location
)
SELECT r.reading_id, r.location AS ambiente, r.noted_date,
       r.temperature AS temp, ROUND(e.media, 2) AS media_ambiente,
       ROUND((r.temperature - e.media) / e.desvio, 2) AS z_score
FROM temperature_readings r
JOIN estatisticas e ON e.location = r.location
WHERE ABS(r.temperature - e.media) > 2 * e.desvio;
```

View 3 - vw_tendencia

Pergunta: como a temperatura evolui ao longo do tempo? **Propósito:** agregar por dia as temperaturas máxima, mínima e média, além do número de leituras, em ordem cronológica. Como a data foi convertida para **TIMESTAMP** no ETL, o agrupamento usa **DATE(noted_date)** e a ordenação é realmente cronológica, e não alfabética. Resultado: 86 dias com leituras.

```
CREATE OR REPLACE VIEW vw_tendencia AS
SELECT DATE(noted_date) AS data_leitura,
       MAX(temperature) AS temp_max, MIN(temperature) AS temp_min,
       ROUND(AVG(temperature), 2) AS temp_media, COUNT(*) AS total_leituras
FROM temperature_readings
GROUP BY DATE(noted_date)
ORDER BY data_leitura;
```

View 4 (complementar) - vw_leituras_por_hora

Pergunta: em que horários os sensores mais registram dados? **Propósito:** contar as leituras e calcular a temperatura média por hora do dia (0 a 23) e por ambiente. Ajuda a avaliar a frequência de coleta e a identificar horários de maior atividade.

```
CREATE OR REPLACE VIEW vw_leituras_por_hora AS
SELECT EXTRACT(HOUR FROM noted_date)::INT AS hora, location AS ambiente,
       COUNT(*) AS contagem, ROUND(AVG(temperature), 2) AS temp_media
FROM temperature_readings
GROUP BY EXTRACT(HOUR FROM noted_date), location;
```

5. Visualizações no Streamlit

A seguir, os prints do dashboard em execução. Todos os gráficos são interativos (Plotly): permitem zoom, ocultar séries clicando na legenda e ver os valores exatos ao passar o mouse.

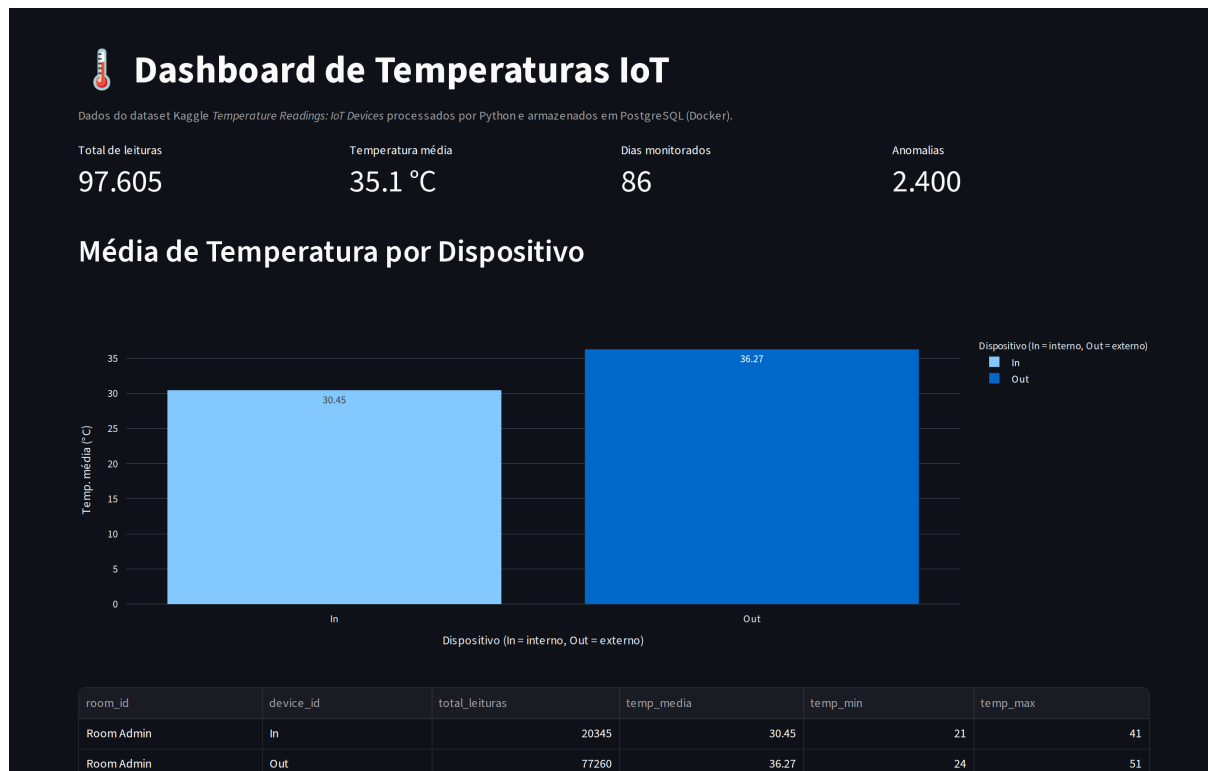


Figura 2 – Indicadores gerais e temperatura média por dispositivo (View 1).

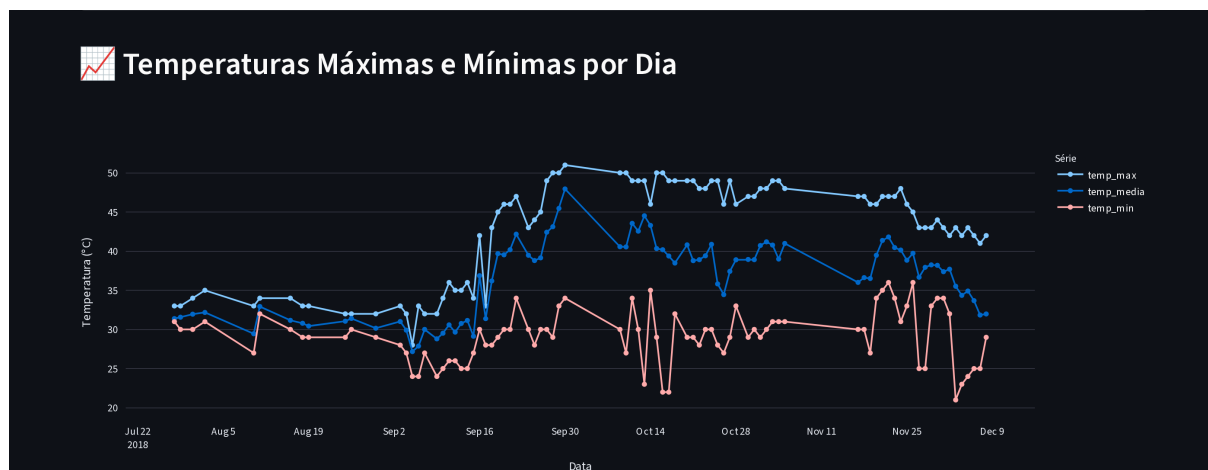


Figura 3 – Temperaturas máximas, médias e mínimas por dia (View 3).



Figura 4 – Anomalias detectadas por z-score, separadas por ambiente (View 2).

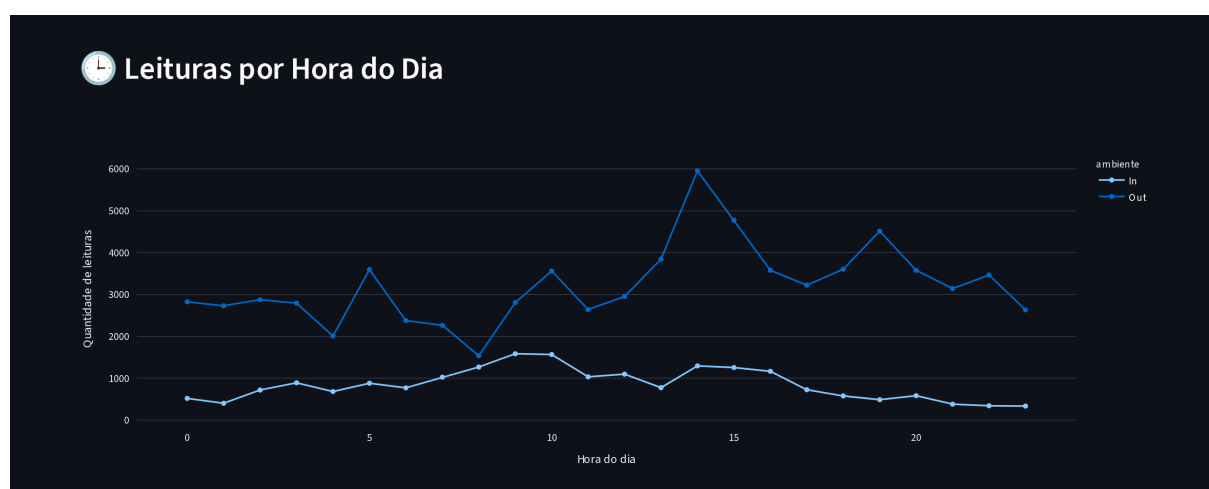


Figura 5 – Quantidade de leituras por hora do dia (View 4).

6. Insights obtidos

- **O ambiente externo é mais quente e bem mais instável.** A média externa é de 36,3 °C, contra 30,5 °C no interno. O desvio-padrão externo (5,7 °C) é mais que o dobro do interno (2,2 °C): o ambiente interno funciona como um amortecedor térmico.
- **Há um salto de temperatura externa a partir de outubro.** A média externa passa de cerca de 31 °C (jul-set) para cerca de 40 °C (out-nov), enquanto a interna quase não muda. Isso indica sazonalidade e boa isolamento do ambiente interno. O pico diário foi de 51 °C (30/09/2018).
- **A coleta é desigual entre os sensores.** 79% das leituras vêm do sensor externo. Em uma análise, isso pode distorcer médias gerais se os ambientes não forem separados, e por isso as views agrupam por ambiente.
- **Anomalias são raras, mas relevantes.** 2,5% das leituras (2.400) estão fora de ± 2 desvios-padrão. No sensor interno aparecem picos de até 41 °C, incomuns para um ambiente fechado, que mereceriam investigação (falha do ar-condicionado ou do sensor).
- **Existem lacunas na coleta.** Há leituras em apenas 86 dos cerca de 134 dias do período, o que indica dias sem transmissão (sensor desligado, falta de energia ou falha de rede).
- **Os horários de coleta variam por sensor.** O sensor externo registra mais leituras por volta das 14h, enquanto o interno concentra suas leituras entre 9h e 10h.

Mês	Média interna (°C)	Média externa (°C)
07/2018	31,1	31,9
08/2018	29,9	31,2
09/2018	30,7	31,2
10/2018	31,5	39,9
11/2018	30,4	40,2
12/2018	28,4	37,3

Tabela – Média mensal por ambiente, destacando o aumento externo a partir de outubro.

7. Uso prático em ambiente real

- **Alertas automáticos:** um job consulta a `vw_anomalias` periodicamente e envia avisos por e-mail, Telegram ou Teams quando surge uma leitura anômala.
- **Automação de climatização:** acionar o ar-condicionado ou a ventilação quando a temperatura externa ultrapassar um limite, antes que o ambiente interno aqueça.
- **Saúde dos sensores:** detectar dias ou horas sem leitura e abrir chamados de manutenção automaticamente.

- **Eficiência energética:** cruzar a tendência diária com o consumo de energia para planejar horários de refrigeração.
- **Escalabilidade:** em produção, a ingestão por CSV seria substituída por streaming (MQTT ou Kafka), com o mesmo PostgreSQL (ou TimescaleDB, para séries temporais) e o dashboard publicado em nuvem.
- **Inteligência Artificial:** com o histórico organizado, seria possível treinar modelos de previsão de temperatura e de detecção de anomalias mais sofisticados.

8. Conclusão

O projeto entregou um pipeline de dados IoT completo e reproduzível: os dados brutos do Kaggle são tratados em Python, armazenados em um PostgreSQL isolado em Docker, organizados em views SQL que respondem a perguntas claras e apresentados em um dashboard interativo. Além de cumprir os requisitos, o trabalho aplicou boas práticas de engenharia de dados: tipagem correta das datas, remoção de duplicados, credenciais fora do código, SQL versionado como fonte única da verdade e histórico de commits no GitHub. Os insights mostram como uma estrutura simples já permite apoiar decisões reais de monitoramento e climatização.

Referências

- Repositório do projeto: <https://github.com/KaiqueDev1/iot-faculdade-setembro>
- Dataset: *Temperature Readings: IoT Devices*, <https://www.kaggle.com/datasets/atulanandjha/temperature-readings-iot-devices>
- Documentação do Streamlit: <https://docs.streamlit.io>
- Documentação do PostgreSQL: <https://www.postgresql.org/docs/>
- Documentação do SQLAlchemy: <https://docs.sqlalchemy.org>
- Documentação do Docker Compose: <https://docs.docker.com/compose/>